

Problem Set 3

Hoa T. Vu

Algorithms

Problem 1: Coin Change. Given coin denominations, find the **minimum number of coins** to make a target amount. Each coin may be used any number of times.

Example: coins = [1, 5, 6, 9], amount = 11. Greedy (largest first) gives $9 + 1 + 1 = 3$ coins. Can you do better?

Hint: think about subproblems. If you know the minimum coins for every amount $< a$, how can you compute the minimum coins for amount a ?

- (a) Define the subproblem. Write the recurrence.
- (b) Implement `coin_change(coins, amount)`.
- (c) What is the running time?

Problem 2: Counting Inversions. An **inversion** in a list is a pair (i, j) with $i < j$ but $\text{arr}[i] > \text{arr}[j]$.

```
arr = [3, 1, 2, 5, 4]
# inversions: (3,1), (3,2), (5,4) -> count = 3
```

- (a) Write a brute-force solution using two nested loops. What is its time complexity?
- (b) (*Bonus*) Write an $O(n \log n)$ solution.

Hint: think about merge sort. When merging two sorted halves, every element in the right half that is placed before an element in the left half contributes exactly (number of remaining elements in the left half) inversions.

Problem 3: Maximum Subarray (Divide and Conquer). Given an array of integers (possibly negative), find the contiguous subarray with the largest sum.

Example: [-2, 1, -3, 4, -1, 2, 1, -5, 4] $\Rightarrow$ subarray [4, -1, 2, 1], sum = 6.

Key idea: split the array at the midpoint. The maximum subarray either lies entirely in the left half, entirely in the right half, or crosses the midpoint. The crossing case can be solved in $O(n)$ by scanning outward from the midpoint.

- (a) Write `max_crossing(a, lo, mid, hi)` that returns the maximum sum of a subarray crossing the midpoint.
- (b) Write `max_subarray(a, lo, hi)` recursively using the above.
- (c) Write the recurrence $T(n)$ and solve it. What is the running time?

Problem 4: Longest Common Subsequence. Given two strings s and t , find the length of their longest common subsequence (LCS). A subsequence does not need to be contiguous.

Example: $s = \text{"abcde"}, t = \text{"ace"} \Rightarrow \text{LCS} = \text{"ace"}$, length 3.

Subproblem: let $\text{dp}[i][j]$ = length of the LCS of $s[:i]$ and $t[:j]$.

Recurrence:

$$\text{dp}[i][j] = \begin{cases} \text{dp}[i-1][j-1] + 1 & \text{if } s[i-1] = t[j-1] \\ \max(\text{dp}[i-1][j], \text{dp}[i][j-1]) & \text{otherwise} \end{cases}$$

- (a) Implement `lcs(s, t)` using bottom-up DP.
- (b) Verify: `lcs("abcde", "ace") = 3`, `lcs("abc", "abc") = 3`, `lcs("abc", "def") = 0`.
- (c) What is the time and space complexity?

Problem 5: 3-Sum. Given an array of integers and a target T , find all unique triplets (a, b, c) such that $a + b + c = T$.

Example: $[-1, 0, 1, 2, -1, -4], T = 0 \Rightarrow$ triplets: $(-1, -1, 2)$ and $(-1, 0, 1)$.

- (a) Write a brute-force $O(n^3)$ solution using three nested loops.
- (b) Write an $O(n^2)$ solution.
Hint: sort the array. For each element $a[i]$, use two pointers `lo = i+1` and `hi = n-1` to find pairs summing to $T - a[i]$. Advance `lo` if the sum is too small, retreat `hi` if too large.
- (c) How do you avoid duplicate triplets in the $O(n^2)$ solution?